Luna Bridge

SNE-SPIMS

Terminology Study Guide

Know every term you'll say (or be asked) in the demo & tech talk

Tech stack • Architecture • Security • Domain terms • Q&A

0. Say this first (memorise it)

The 20-second description of the whole project:

> **Elevator pitch**
> “SNE-SPIMS is a web application for a special-needs school. Parents, teachers, and an admin share one system, but each role only sees what's relevant to them. It's built with Spring Boot and renders pages on the server with Thymeleaf; data lives in a MySQL database accessed through MyBatis. Every piece of information is tied to a child, and access is controlled by role.”

What the letters mean

SNE	Special-Needs Education — schooling adapted for students with disabilities.
SPIMS	Student Progress & Information Management System — the kind of system this is.
Luna Bridge	The school's brand name shown in the UI (logo, login). Same product as SNE-SPIMS.

1. Tech-stack terms

Java	A widely-used, statically-typed programming language that runs on the JVM (Java Virtual Machine). *In this app: The whole backend is Java; built to target Java 17, runs on a newer JDK.*
Spring Boot	The most popular Java framework for building web apps; auto-configures everything and bundles a web server so the app runs as a single program. *In this app: Version 3.3.5 — the foundation of the app; starts an embedded Tomcat server on port 8080.*
Spring MVC	The part of Spring that handles web requests using the Model-View-Controller pattern. *In this app: Maps URLs like /dashboard to Java methods (controllers) that return a page.*
Spring Security	Spring's library for login, passwords, and access control. *In this app: Provides the login form, checks passwords, and enforces which role can open which URL.*
Thymeleaf	A server-side HTML template engine: HTML files with special th:* attributes that get filled with real data before being sent to the browser. *In this app: Every page (dashboard, login, messages…) is a Thymeleaf template.*
Thymeleaf Layout Dialect	An add-on that lets all pages share one common layout (header, side rail) instead of repeating it. *In this app: layout/main.html is the shared shell every page 'decorates'.*

MyBatis	A 'SQL mapper' — you write the SQL yourself in XML files and it maps the results to Java objects. (Lighter than a full ORM like JPA/Hibernate.) *In this app: All database queries live in mappers/*.xml; used standalone, wired by hand.*
MySQL	A popular open-source relational database (data stored in tables with rows and columns). *In this app: Stores users, students, IEPs, reports, sessions, messages, etc.*
MariaDB / XAMPP	MariaDB is a drop-in MySQL replacement; XAMPP is an easy bundle that installs MySQL/MariaDB + tools on a PC for local development. *In this app: The dev database runs under XAMPP locally.*
HikariCP	A connection pool — it keeps a set of open database connections ready to reuse, instead of opening a new one each time (much faster). *In this app: One shared pool; MyBatis borrows connections from it.*
JDBC	Java Database Connectivity — the standard Java API for talking to SQL databases; the driver is the database-specific plug-in. *In this app: The MySQL JDBC driver is what actually sends SQL to the database.*
BCrypt / jBCrypt	A secure, slow hashing algorithm designed for passwords; jBCrypt is its Java implementation. *In this app: Passwords are stored as BCrypt hashes ($2a$…), never as plain text.*
Maven	A build tool for Java — it downloads libraries and compiles/packages the app. *In this app: Bundled Maven builds and runs the app (mvn spring-boot:run).*
Bootstrap	A popular CSS framework of ready-made UI components (buttons, dropdowns, grids). *In this app: Used for base components; restyled with the app's warm 'clay' colour theme.*
Lexend	A Google font designed to reduce reading fatigue — good for accessibility. *In this app: The app's UI font.*
oklch	A modern way to define colours by lightness, chroma (saturation), and hue — easy to keep a palette consistent. *In this app: All design colours/tokens in app.css are written in oklch.*

2. Architecture & backend terms

Layered architecture	Organising code in layers, each with one job, calling only the layer below: Controller → Service → DAO → Database. *In this app: Keeps web code, business rules, and SQL cleanly separated.*

MVC (Model-View-Controller)	A pattern: the Controller handles the request, the Model is the data, the View is the page shown. *In this app: Controller (Java) + Model (data) + View (Thymeleaf HTML).*
Controller	A class that receives a web request and decides what to do, then returns which page to show. *In this app: e.g. DashboardController handles GET /dashboard.*
Service layer	Where the business rules live (the 'what should happen' logic), independent of the web or database. *In this app: e.g. ReportService, AccessControl — also where access rules are enforced.*
DAO (Data Access Object)	A class whose only job is to read/write one type of data in the database. *In this app: e.g. StudentDao — its SQL lives in StudentMapper.xml.*
Model / POJO	A 'Plain Old Java Object' — a simple class holding fields that mirror a database row. *In this app: e.g. Student, DailyReport, IepGoal.*
Dependency Injection (DI)	Instead of a class creating the objects it needs, the framework hands them in. Makes code easier to test and swap. *In this app: Spring injects services into controllers; objects called 'beans'.*
Bean	Any object that Spring creates and manages for you. *In this app: Services are exposed as beans so controllers can use them.*
Service locator / Singleton	Singleton = exactly one shared instance of a class. A service locator is one object that hands out all the others. *In this app: A hand-wired Services locator builds the DAOs/services once; a small config bridges them into Spring.*
Connection pool	A ready set of reusable database connections (see HikariCP). *In this app: Avoids the cost of opening a new DB connection per request.*
Server-side rendering (SSR)	The server builds the full HTML page and sends it ready-to-display (vs. a JavaScript app that builds pages in the browser). *In this app: Why the app feels simple and has no separate front-end build.*
Template engine	The tool that merges data into an HTML template (see Thymeleaf).
Application event	An in-app message: one part 'publishes' an event, another 'listens' and reacts — keeps features decoupled. *In this app: Publishing a daily report fires an event that creates a parent notification.*

3. Web & security terms

HTTP	The protocol browsers use to talk to servers, via requests and responses.

GET vs POST	GET asks for a page/data; POST submits data (a form) that changes something. *In this app: GET /dashboard shows a page; POST /messages/send sends a chat message.*
Endpoint / route	A specific URL the server responds to. *In this app: e.g. /students, /reports/new.*
Session	The server's memory that you're logged in, kept across requests via a cookie. *In this app: Keeps you signed in as you move between pages.*
Cookie	A small piece of data the browser stores and sends back to the server (e.g. your session id).
Authentication vs Authorization	Authentication = proving who you are (login). Authorization = what you're allowed to do. *In this app: Login authenticates; role checks authorize.*
Role-based access control (RBAC)	Permissions are granted by role, not per-person. *In this app: Three roles: ADMIN, TEACHER, PARENT — each sees different menus/data.*
CSRF	Cross-Site Request Forgery — an attack where another site tricks your browser into submitting a form; a hidden token blocks it. *In this app: Every form includes a CSRF token; Spring Security checks it.*
Form login	Logging in with a username/password HTML form (vs. tokens/SSO). *In this app: The /login page; passwords verified against BCrypt hashes.*
CSS variables / design tokens	Named, reusable values (colours, spacing) defined once and used everywhere. *In this app: app.css defines --red, --surface, etc. — change once, applies app-wide.*
Favicon	The small icon shown on the browser tab. *In this app: Set to the Luna Bridge logo.*

4. Domain terms (special-needs education)

IEP (Individualised Education Program)	A formal, personalised plan describing a student's goals, current levels, and the accommodations they need. *In this app: The long-term progress tracker; each student has one active IEP.*
IEP goal	A single target within an IEP, in a development area, with a progress percentage. *In this app: Updated over time; the history drives the progress trend.*
Development domain / area	The five areas progress is grouped into: Academic, Communication, Social/Emotional, Self-Care, Motor. *In this app: Each has its own colour in the UI.*

Daily report	A short note a teacher writes about a student's day (summary, mood, ratings). *In this app: Published to parents, who can 'acknowledge' (mark as read).*
Rubric	A set of criteria with rating levels used to assess consistently (e.g. Emerging → Developing → Proficient). *In this app: Each daily report scores a few rubric criteria.*
Therapy session	A scheduled specialist session (speech, occupational, physical, counselling) in a room. *In this app: Scheduling runs a 3-way clash check: student, therapist, and room.*
Behaviour incident	A recorded event of concerning behaviour, with a severity and the action taken. *In this app: Severity LOW / MEDIUM / HIGH; a 'parent-visible' switch controls if families see it.*
Caseload / roster	Caseload = the students a teacher is responsible for. Roster = the full list of students. *In this app: A teacher only sees their caseload; admin sees the whole roster.*
Announcement	A school notice targeted at everyone, teachers, or parents. *In this app: Created by admin; shown as a dashboard widget.*
Notification	A personal alert (bell icon) — new report, therapy reminder, new message. *In this app: Clicking one deep-links to the exact page.*
Acknowledge	A parent confirming they've read a report. *In this app: Marks the report read and notifies the teacher.*

5. Likely questions — short answers

Read these once; they cover the questions an audience usually asks.

Q: Why MyBatis instead of JPA/Hibernate?
A: We wanted full control over the SQL and a lighter data layer. MyBatis lets us write and tune the exact queries, with results mapped to Java objects — simpler to reason about for this project.

Q: Why server-side rendering instead of React/Angular?
A: One codebase, no separate front-end build, and security is enforced in one place on the server. Thymeleaf was the pragmatic choice for a content-driven school app.

Q: How are passwords kept safe?
A: They're never stored in plain text — only as BCrypt hashes, a slow algorithm built for passwords. Login re-hashes the input and compares.

Q: How do you stop one user seeing another's data?
A: Access is enforced in the service layer by a single AccessControl check: admins see everything, teachers only their caseload, parents only their own children. The UI also hides what you can't access, but the server is the real gate.

Q: What happens if two therapy sessions overlap?
A: Scheduling runs a three-way conflict check — the same student, therapist, or room can't be double-booked at the same time; the save is rejected with a clear message.

Q: Is it scalable / production-ready?
A: The architecture is clean and layered, with a connection pool and stateless request handling, so it can scale. For production we'd externalise the database credentials, add HTTPS, and run behind a proper server — it's currently tuned for a local demo.

Q: Which database, and can it change?
A: MySQL (MariaDB under XAMPP in development). Because all SQL is isolated in MyBatis mapper files and we use standard JDBC, switching databases mostly means swapping the driver and adjusting SQL.

Q: What are the main features?
A: Role dashboards, IEP goals & progress, daily reports with parent acknowledgement, therapy scheduling with conflict checks, behaviour incidents, parent–teacher chat, a shared calendar, announcements, and notifications.

Q: How is the look kept consistent?
A: A design system: colours/spacing are CSS 'tokens' in one file (app.css) and every page shares one layout shell, so a single change updates the whole app.

Q: What would you improve with more time?
A: Automated tests coverage, real image assets, HTTPS, externalised secrets, and possibly a richer reporting/analytics view.